

ai_agent_report

May 2026

1 Introduction & Architecture

This project presents an Automated Multimodal Research AI Agent. The system automates a 3-day pipeline to discover, synthesize, and deliver high-signal research updates across academic literature (Arxiv) and technical media (YouTube/Bilibili). This agent introduces a hierarchical data compaction workflow and cross-platform contradiction checking, automatically merges redundant information, alerts users to discrepancies between marketing hype and empirical data, and persists insights into a vector database for interactive follow-up Q&A.

The proposed autonomous research agent adheres to a decoupled, modular engineering paradigm designed to ensure rigid separation of concerns across four sequential layers, with the complete implementation open-sourced on GitHub.¹ And its operational pipeline demonstrated in an accompanying video.²

First, the IR Layer (`tools.py`) executes concurrent dual-track queries via the Arxiv Atom API and Tavily Search API to gather the latest paper abstracts and multimedia metadata from YouTube and Bilibili, utilizing an automated `ARXIV_FALLBACK` routing mechanism to guarantee uptime if the primary Arxiv API times out. Second, the Skill Orchestration Layer (`skills/`) optimizes the context window by abstracting heavy web scraping and multimedia parsing away from the core agent logic; specifically, a baseline video information retrieval skill³ was adapted and significantly extended to natively ingest and process heterogeneous Arxiv literature data under local exception gating. Third, the Reasoning and Compaction Layer (`agent.py`) leverages an OpenAI-compatible DeepSeek API wrapper to extract structured innovation points using Pydantic schemas, execute cross-source factual contradiction checks with explicit confidence scoring, and merge redundant data into centralized thematic nodes based on semantic affinity. Finally, the Memory and Action Layer (`main.py`, `memory/`) processes incoming summaries via stable SHA-256 hashing to commit them alongside their metadata into a persistent ChromaDB database for citation-grounded retrieval-augmented generation (RAG) chatting, while utilizing an idempotent routine under `.state/` to suppress redundant email notifications via SMTP.

¹https://github.com/yhanqi0611-create/ai_agent_ir

²https://drive.google.com/file/d/1GFpF70olcn3Ru0JqZ8DHS9Q6JZfFJRZN/view?usp=drive_link

³<https://summarize.sh>

2 Advanced IR Techniques & Innovation

To mitigate cross-platform redundancy, the agent implements a two-tier compaction pipeline. First, raw streams are distilled into structured Pydantic objects containing bounded innovation claims and explicit evidential anchors. Second, the agent groups these items by semantic affinity rather than superficial keyword matching. Duplicate disclosures—such as a YouTube video and an Arxiv preprint discussing the same repository—are compressed into a single, high-density thematic node, filtering out low-signal media hype.

The system’s primary novelty lies in its algorithmic self-correction and cross-source conflict resolution framework. Before final synthesis, the agent cross-examines peer-reviewed Arxiv abstracts against colloquial social media content from YouTube and Bilibili. This logic is tuned to detect factual discrepancies and numerical inconsistencies, such as exaggerated benchmark claims made in technical talks versus empirical metrics reported in matching literature.

3 Reflection on Working with AI

The development of this autonomous agent relied heavily on a collaborative human-AI pair-programming workflow leveraging the Cursor IDE and the DeepSeek API.

A strict critical verification process was required to mitigate systemic AI limitations, such as code hallucination and architectural blindness. For instance, initial AI-generated code frequently attempted to pass raw, unfiltered video metadata straight into the core language model’s context window, ignoring token constraints and structural API limits. Human intervention was vital to enforce software engineering decoupling—specifically, by guiding the AI to design the modular `SkillManager` layer and the isolated exception-gating routine within `skills/moltbody.summarize.py`. Furthermore, the AI-generated fallback routines for the Arxiv API initially introduced typing mismatches with Pydantic schemas, necessitating explicit manual refactoring and edge-case debugging.

Ultimately, this experience demonstrates that while modern AI tools are exceptionally proficient at tactical execution, strategic architectural robustness, systemic safety gating, and critical output verification remain the exclusive domain of human engineers. This lab underscores that understanding every layer of a co-created system is paramount to maintaining architectural integrity.